MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

SELETIVA AULA 10

Geometria computacional

Busca Ternária

JP <jpds>

Pou <jvgc>

rev. 1.1

CC BY NC SA

MaratonaCIn

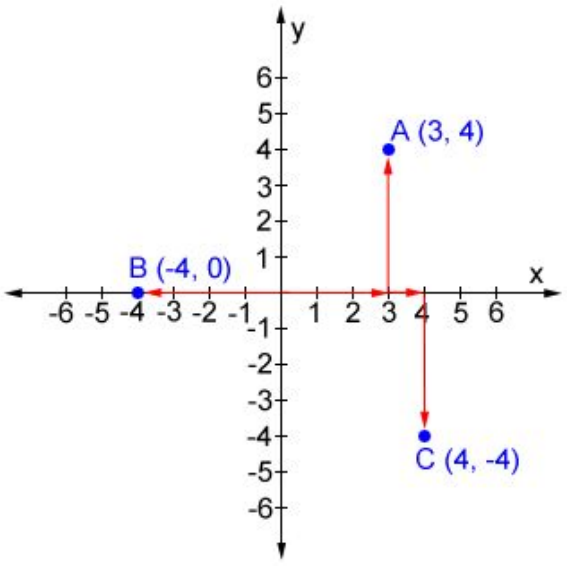
Centro de Informática UFPE

Universidade Federal de Pernambuco

Primitivas

Ponto

- Representa uma posição no plano: (x, y)
- Não possui tamanho, apenas localização
- Base para construção de outras estruturas

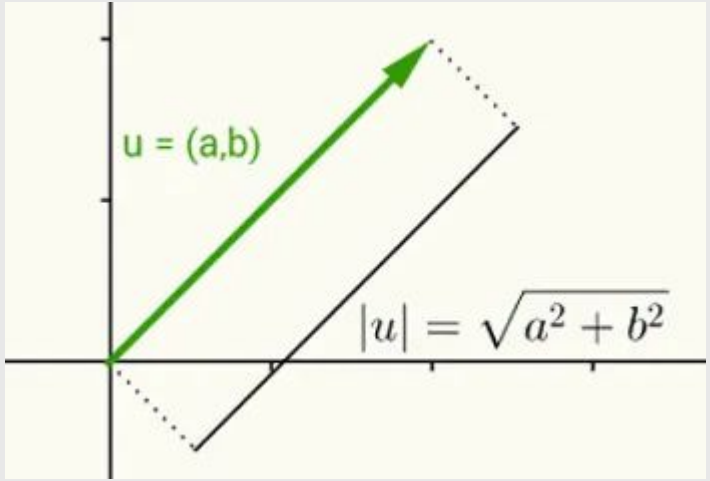


Vetor

- Representa um deslocamento ou uma magnitude com direção.

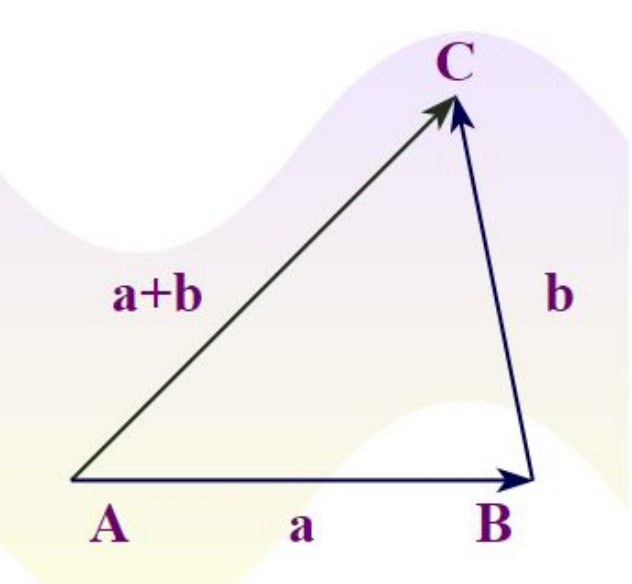
- Representamos da mesma forma que pontos

- Vetor = Ponto no qual o vetor termina com sua origem no (0,0)



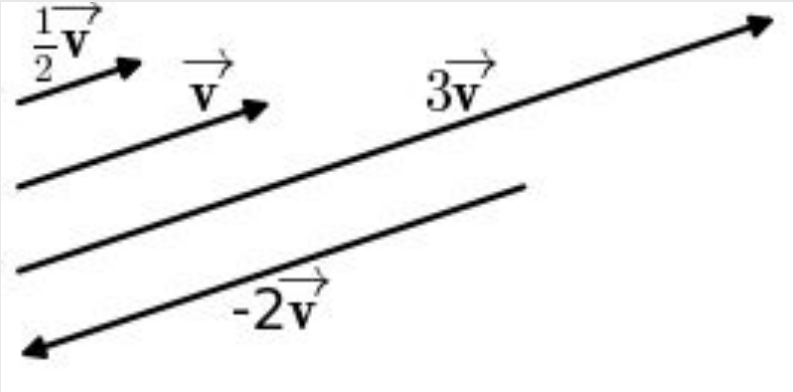
Soma

- Soma componente a componente
- a + b = (ax + bx, ay + by)
- Interpretação: seguir um vetor depois do outro



Multiplicação por escalar

- Escala o tamanho do vetor
- k·v = (k·x, k·y)
- Inverte o sentido se k < 0



Código

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

```
struct Point{
    double x,y;
    Point(double x = 0, double y = 0) : x(x),y(y){}
    Point(const Point& p): x(p.x), y(p.y){}
  bool operator < (const Point &p) const {
    if(cmp(x, p.x) != 0) return x < p.x;
    return cmp(y, p.y) < 0;
  }
  bool operator == (const Point &p) const {return !cmp(x, p.x) && !cmp(y, p.y);}
  bool operator != (const Point &p) const {return !(p == *this);}

    // basic ops
    Point operator + (const Point& p) const {return Point(x+p.x,y+p.y);}
    Point operator - (const Point& p) const {return Point(x-p.x,y-p.y);}
    Point operator * (const double k) const {return Point(x*k,y*k);}
    Point operator / (const double k) const {return Point(x/k,y/k);}
};
```

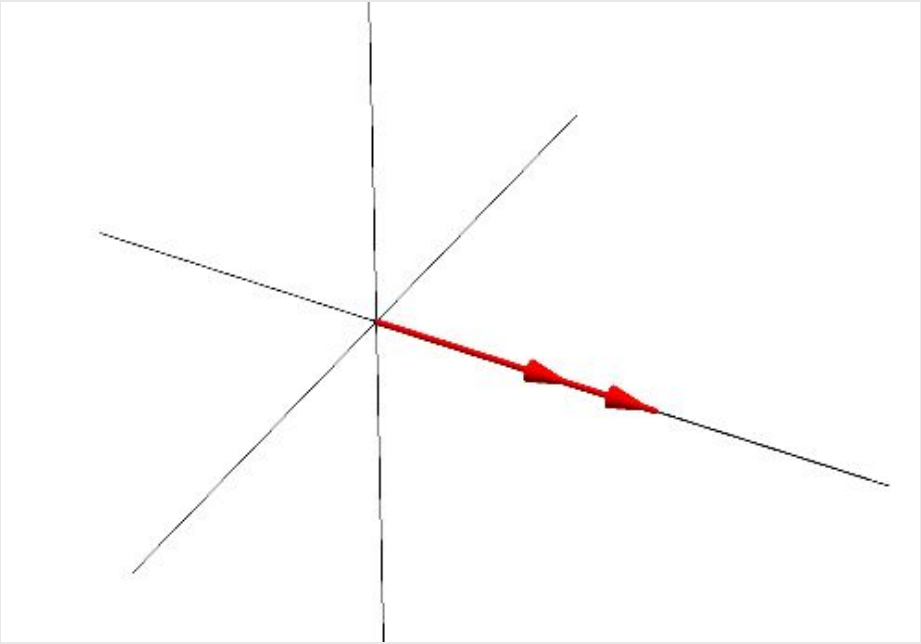
Dot Product

$$Cos(\theta) = \frac{a.b}{|a||b|}$$

- Mede alinhamento entre vetores
- $a \cdot b > 0$ → mesmo sentido
- $a \cdot b = 0$ → perpendiculares
- $a \cdot b < 0$ → sentidos opostos

Cross Product

- Checar posição
- $a \times b > 0 \rightarrow$ anti-horário
- $a \times b < 0 \rightarrow$ horário
- Resulta no vetor perpendicular à ambos
- Módulo do vetor é igual à área do paralelogramo



Código

```
double dot (const Point& p,const Point& q) { return p.x*q.x + p.y*q.y; }
double cross (const Point& p,const Point& q) { return p.x*q.y - p.y*q.x; }
double norm(const Point& p) { return hypot(p.x,p.y); }
double dist(const Point& p, const Point& q) { return hypot(p.x-q.x,p.y-q.y); }
double dist2(const Point& p, const Point& q) { return dot(p-q,p-q); }
Point normalize(const Point &p) { return p/hypot(p.x, p.y); }
double angle (const Point& p, Point& q) { return atan2(cross(p, q), dot(p, q)); }
double angle (const Point& p) { return atan2(p.y, p.x); }
```

Lightning Round!

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Como saber se dois vetores são colineares?

Como calcular a área do triângulo formado entre dois vetores

Dados 3 pontos, como saber se eles são colineares?

Problema de Geometria básico

Time limit: 1.00 s **Memory limit:** 512 MB

There is a line that goes through the points $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$. There is also a point $p_3 = (x_3, y_3)$.

Your task is to determine whether p_3 is located on the left or right side of the line or if it touches the line when we are looking from p_1 to p_2.

Input

The first input line has an integer t: the number of tests.

After this, there are t lines that describe the tests. Each line has six integers: x_1, y_1, x_2, y_2, x_3 and y_3.

Output

For each test, print "LEFT", "RIGHT" or "TOUCH".

Constraints

- $1 \leq t \leq 10^5$
- $-10^9 \leq x_1, y_1, x_2, y_2, x_3, y_3 \leq 10^9$
- $x_1 \neq x_2$ or $y_1 \neq y_2$

Example

Input:

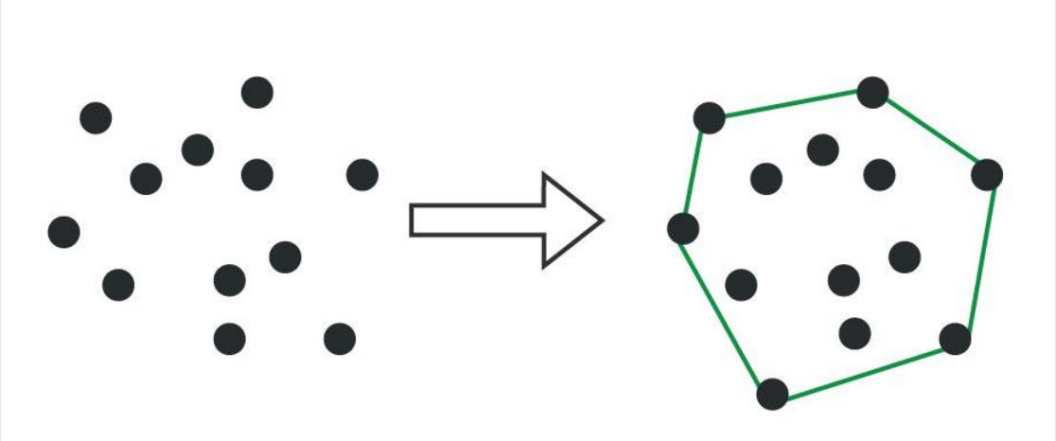
```
3
1 1 5 3 2 3
1 1 5 3 4 1
1 1 5 3 3 2
```

Output:

```
LEFT
RIGHT
TOUCH
```

Convex Hull

Dado um conjunto de pontos, ache o menor polígono convexo que abrange todos os pontos



Código

MaratonaCIn

Centro de Informática UFPE

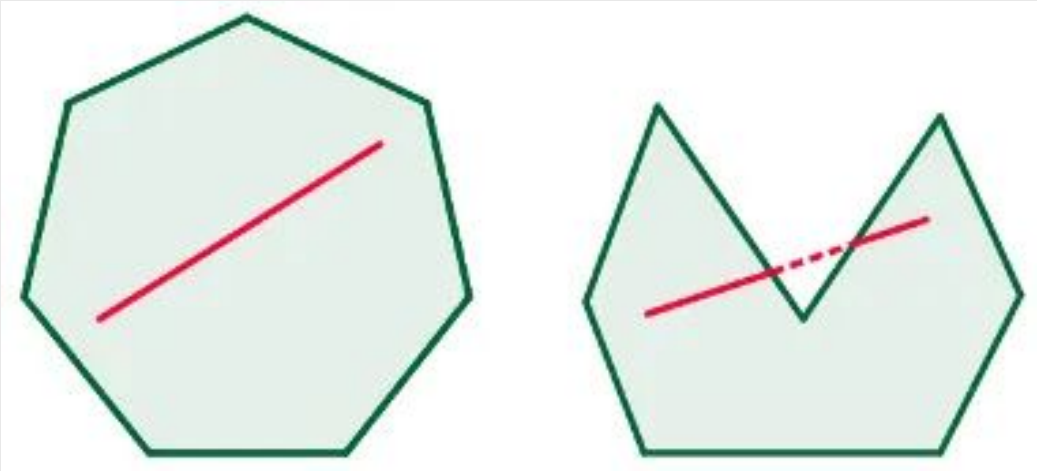
Universidade Federal de Pernambuco

```cpp
vector<Point> convexHull(vector<Point>& pts, bool sorted = false){ // pts.size() >= 3
  int n = pts.size();
  if(!sorted){
    sort(begin(pts),end(pts));
  }
  vector<Point> lower(n + 1), upper(n + 1);
  int s = 0;
  for(int i = 0; i < n; i++){
    lower[s++] = pts[i];
    while(s >= 3){
      Point a = lower[s-3], b = lower[s-2], c = lower[s-1];
      Point v1 = b-a, v2 = c-b;
      if(cross(v1,v2) > 0){
        break;
      }
        lower[s-2] = lower[s-1];
        s--;

    }
  }
  lower.resize(s);
  s = 0;
  for(int i = 0; i < n; i++){
    upper[s++] = pts[i];
    while(s >= 3){
      Point a = upper[s-3], b = upper[s-2], c = upper[s-1];
      Point v1 = b-a, v2 = c-b;
      if(cross(v1,v2) < 0){
        break;
      }
        upper[s-2] = upper[s-1];
        s--;
    }

  }
  upper.resize(s-1);
  reverse(begin(upper),end(upper));
  upper.pop_back();
  lower.insert(end(lower),begin(upper),end(upper));
  return lower;
}
```

Polígonos Côncavos e Convexos

Por quê preferimos/utilizamos polígonos convexos?

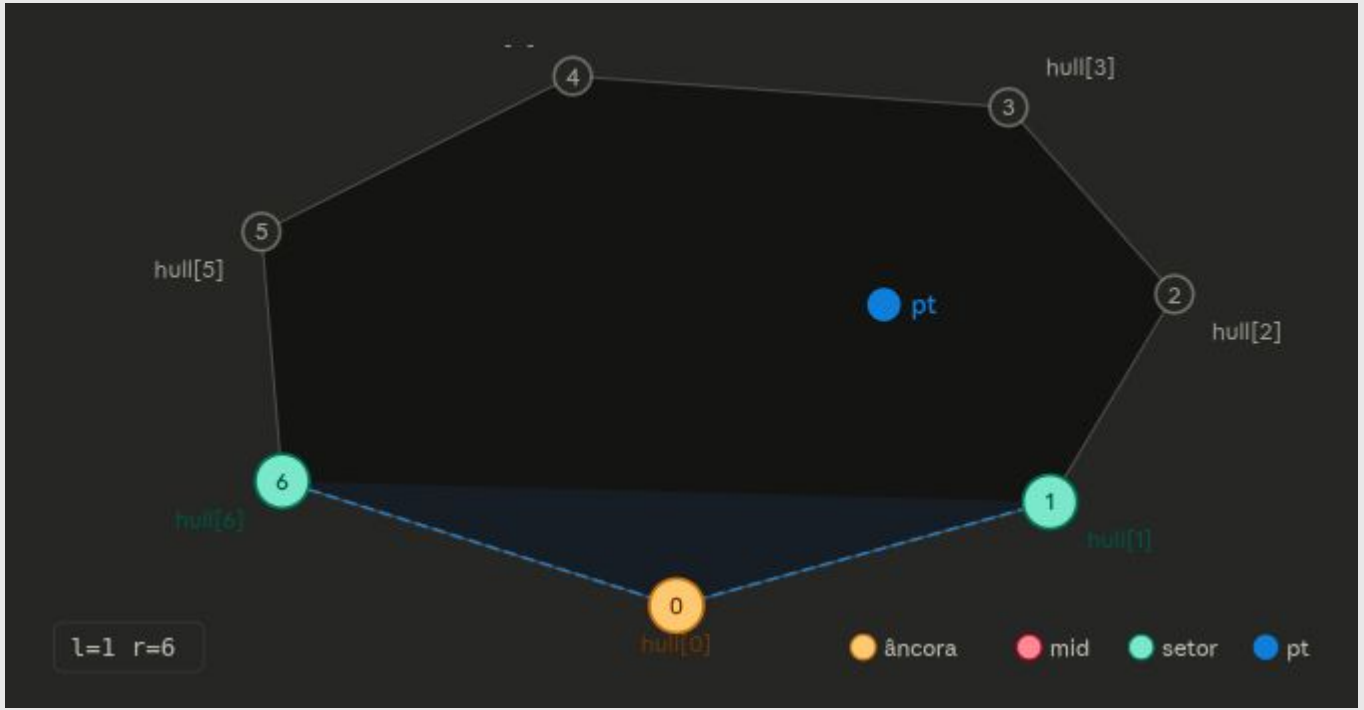
Polígonos Convexos facilitam as buscas

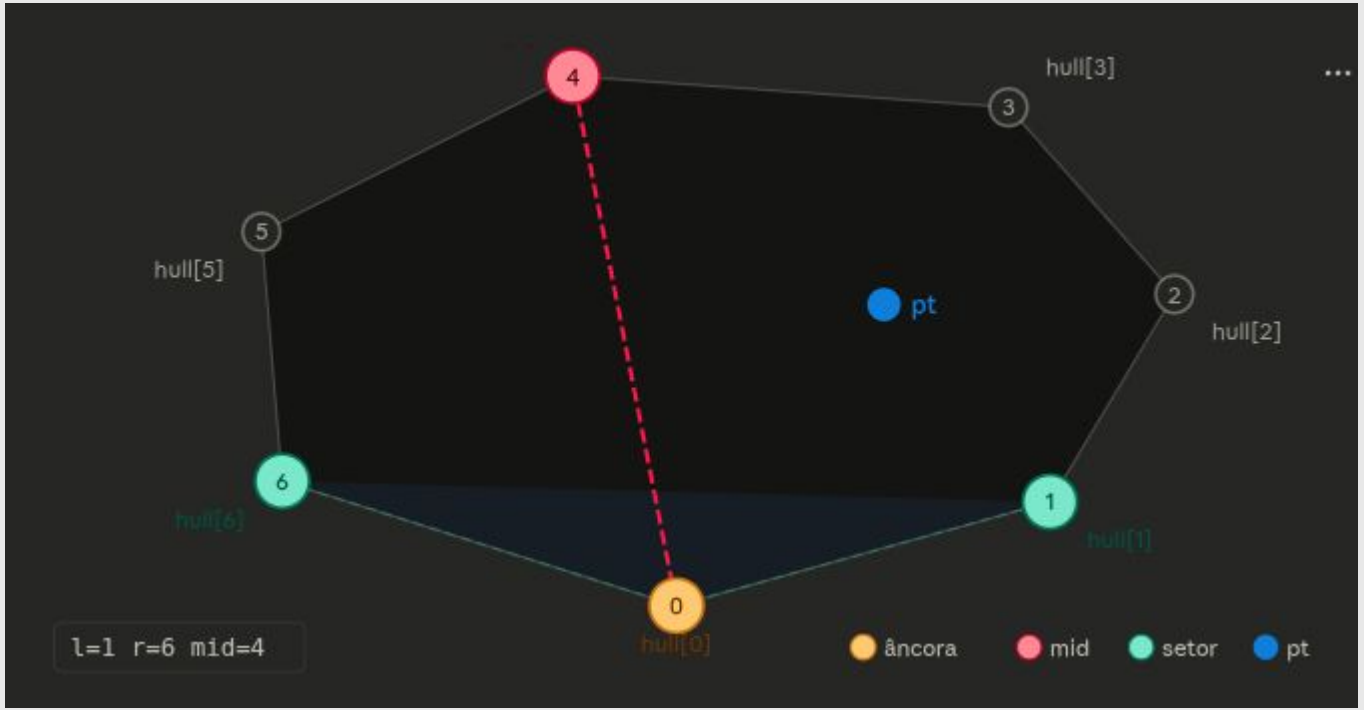


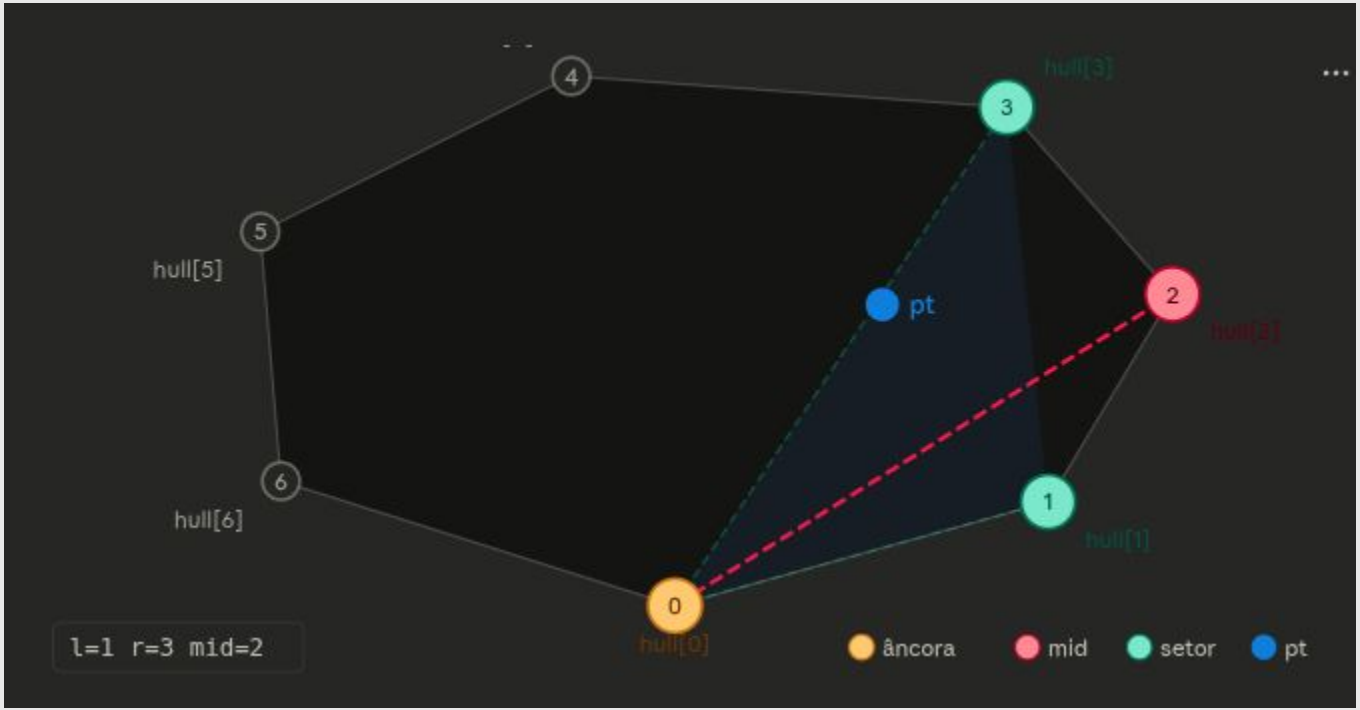
👍

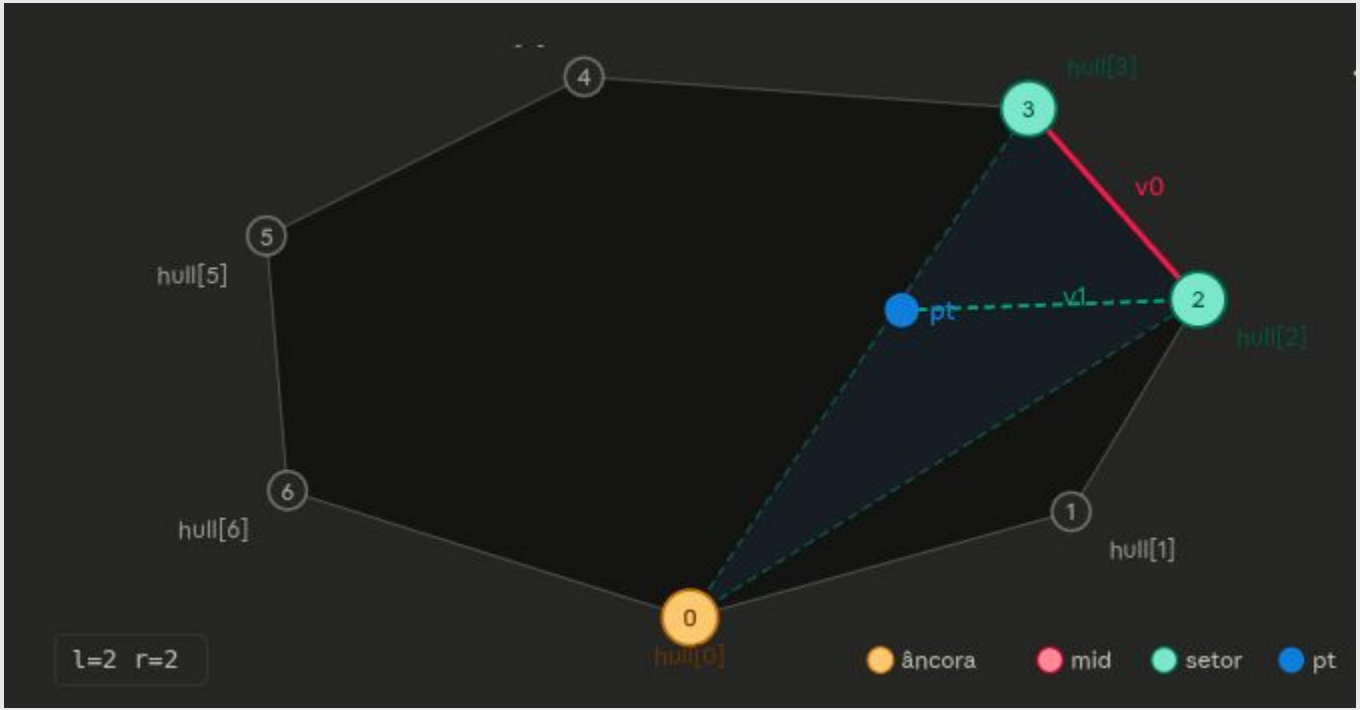
Busca no Convex Hull

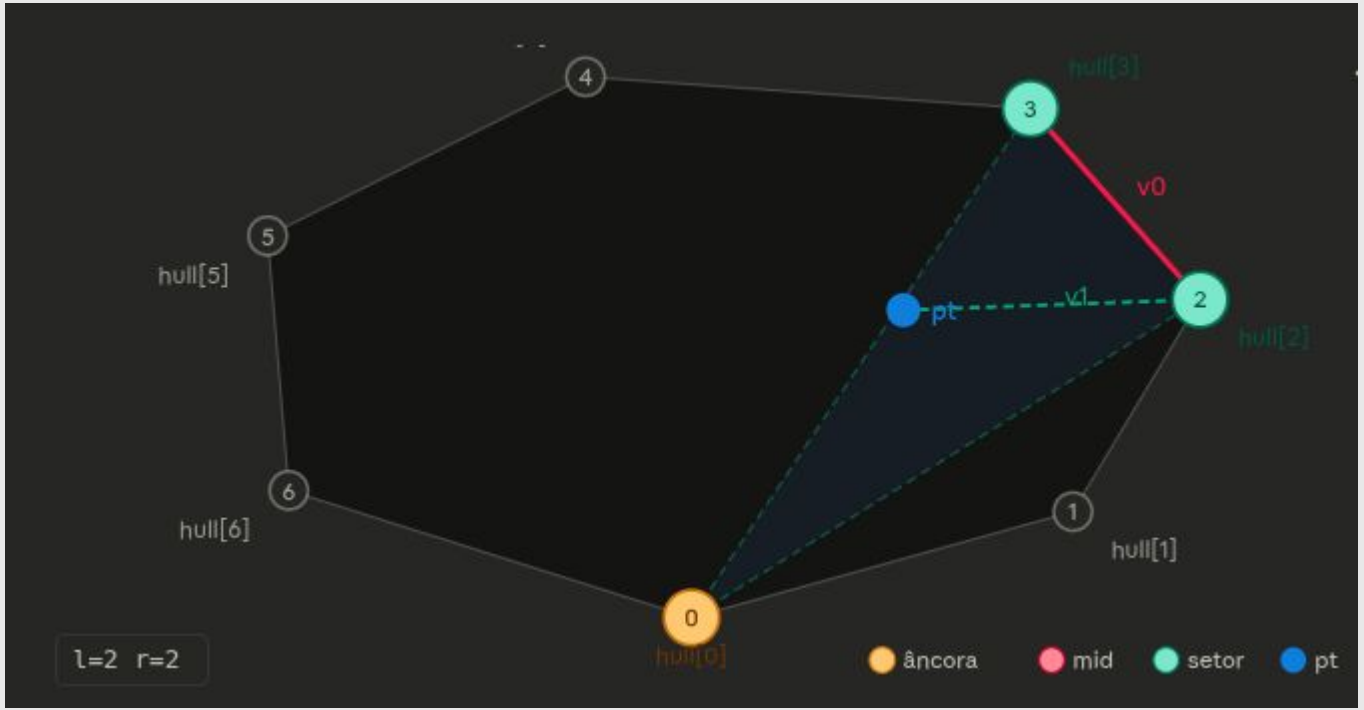
Dado um ponto qualquer, responda se ele está dentro de um polígono


hull[3]
hull[5]
hull[2]
pt
hull[6]
hull[1]
hull[0]
l=1 r=6
âncora
mid
setor
pt


4
hull[3]
3
5
hull[5]
pt
2
hull[2]
6
hull[6]
1
hull[1]
0
hull[0]
l=1 r=6 mid=4
âncora
mid
setor
pt


4
3
hull[3]
5
hull[5]
pt
2
hull[2]
6
hull[6]
1
hull[1]
0
hull[0]
l=1 r=3 mid=2
âncora
mid
setor
pt


4
3
hull[3]
v0
5
hull[5]
pt
v1
2
hull[2]
6
hull[6]
1
hull[1]
0
hull[0]
l=2 r=2
âncora
mid
setor
pt


4
3
hull[3]
v0
5
hull[5]
pt
v1
2
hull[2]
6
hull[6]
1
hull[1]
0
hull[0]
l=2 r=2
âncora
mid
setor
pt

Código

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

```
bool isInside(const vector<Point> &hull, Point pt) {
  int n = hull.size();
    Point v0 = pt - hull[0], v1 = hull[1] - hull[0], v2 = hull[n-1] - hull[0];
  if(cross(v0,v1) > 0 || cross(v0,v2) < 0) {
    return false;
  }
  int l = 1, r = n - 1;
  while(l != r) {
    int mid = (l + r + 1) / 2;
        Point v0 = pt - hull[0], v1 = hull[mid] - hull[0];
    if(cross(v0,v1) < 0) {
      l = mid;
    } else {
      r = mid - 1;
    }
  }
    v0 = hull[(l+1)%n] - hull[l], v1 = pt - hull[l];
  return cross(v0,v1) >= 0;
}
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Busca Ternária

Busca Ternária

Busca ternária é um algoritmo de busca para encontrar o mínimo/máximo de funções unimodais de maneira eficiente.

Uma função unimodal possui um único extremo, e é monotônica para ambos os lados do pico.
O intervalo [l, r] é avaliado em dois pontos igualmente espaçados:

- $m_1 = l + (r - l)/3$
- $m_2 = r - (r - l)/3$

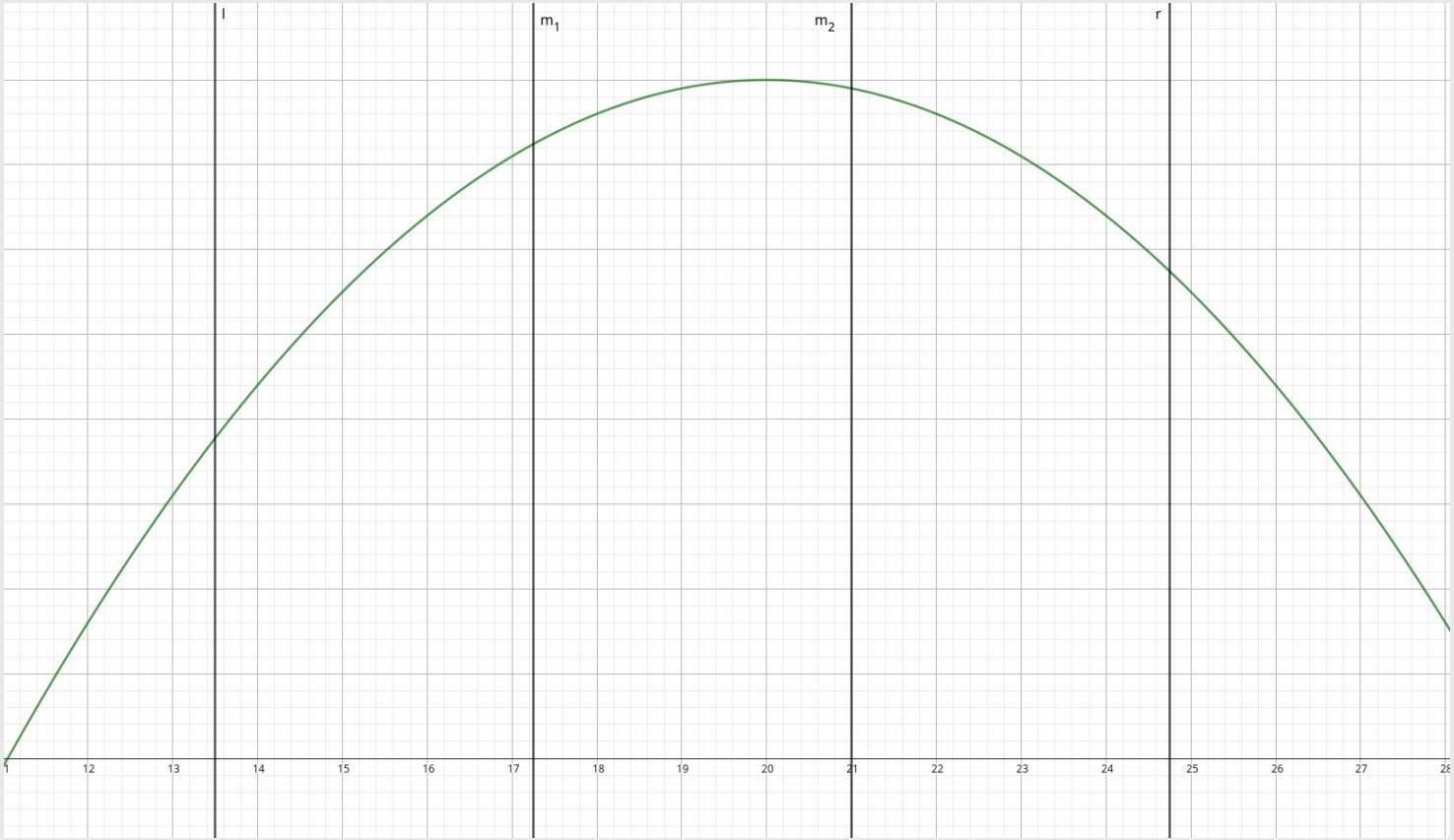
A partir do resultado de f(m1) e f(m2), descartamos ⅓ do intervalo original.

Busca Ternária

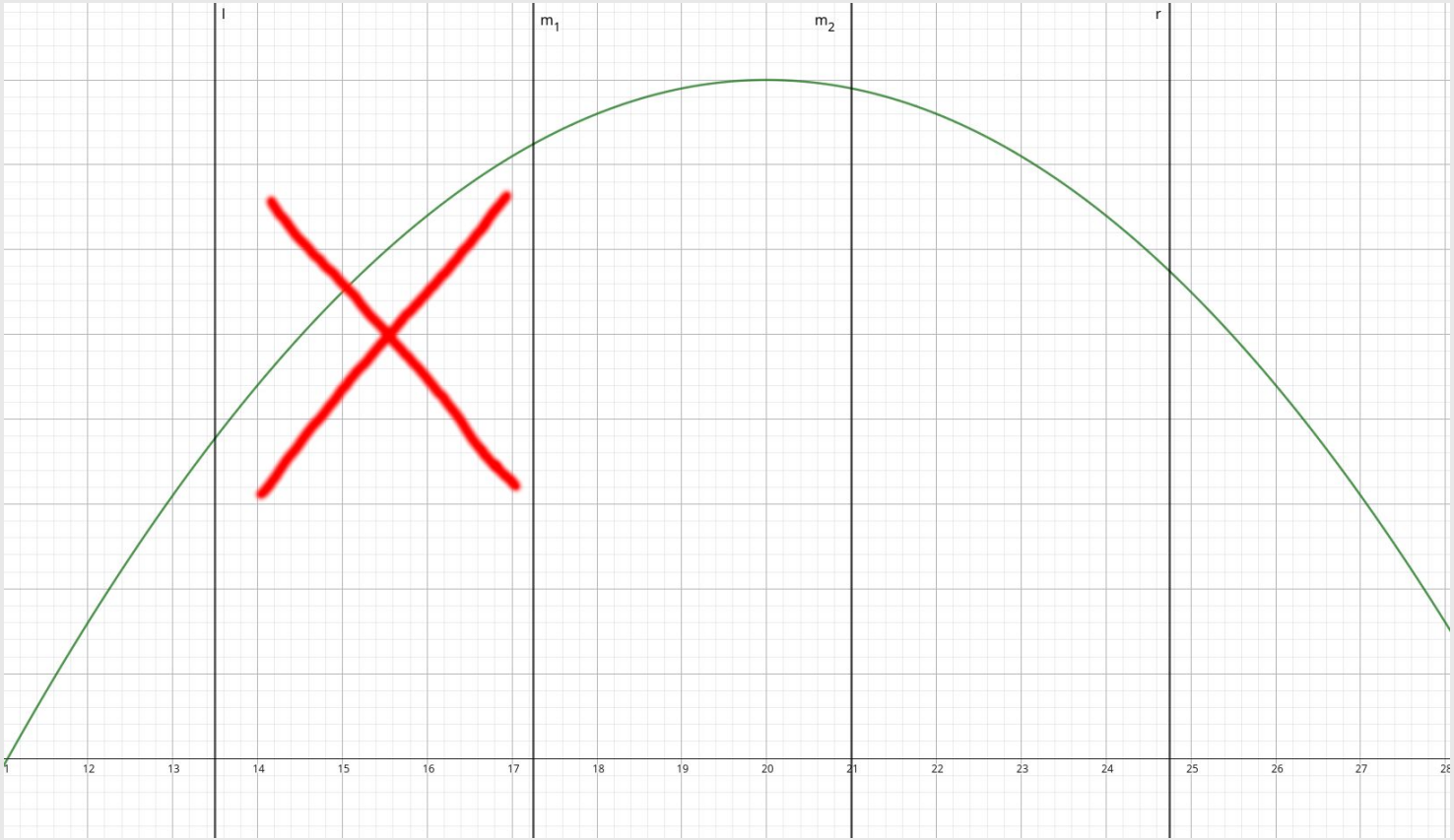
Assumindo que estamos procurando o máximo da função:

- Se f(m1) < f(m2):
  - O máximo não está no intervalo [l, m1], logo, podemos descartá-lo
  - l ← m1
- Senão:
  - O máximo não está no intervalo [m2, r], logo, podemos descartá-lo
  - r ← m2

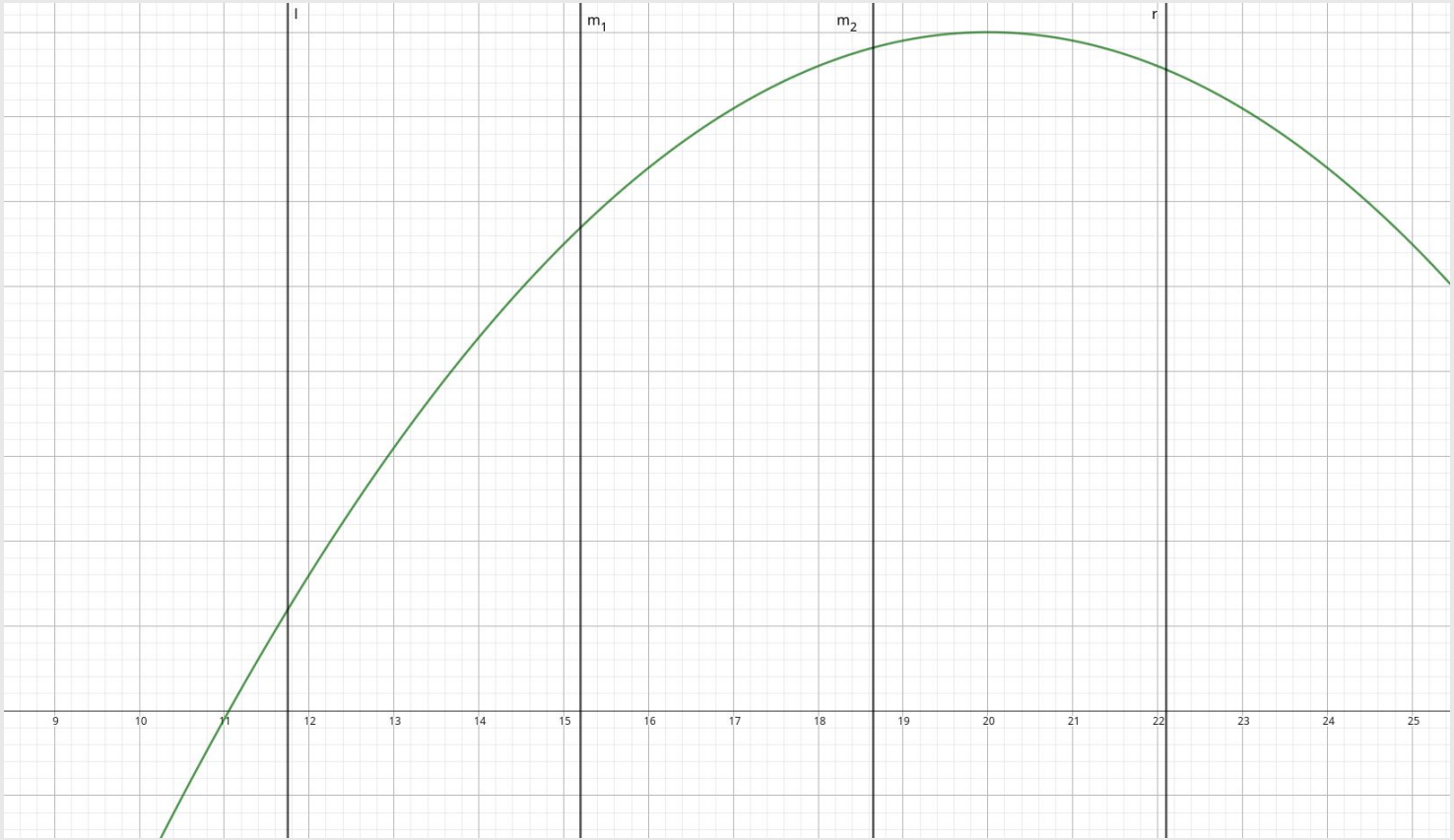
Busca Ternária



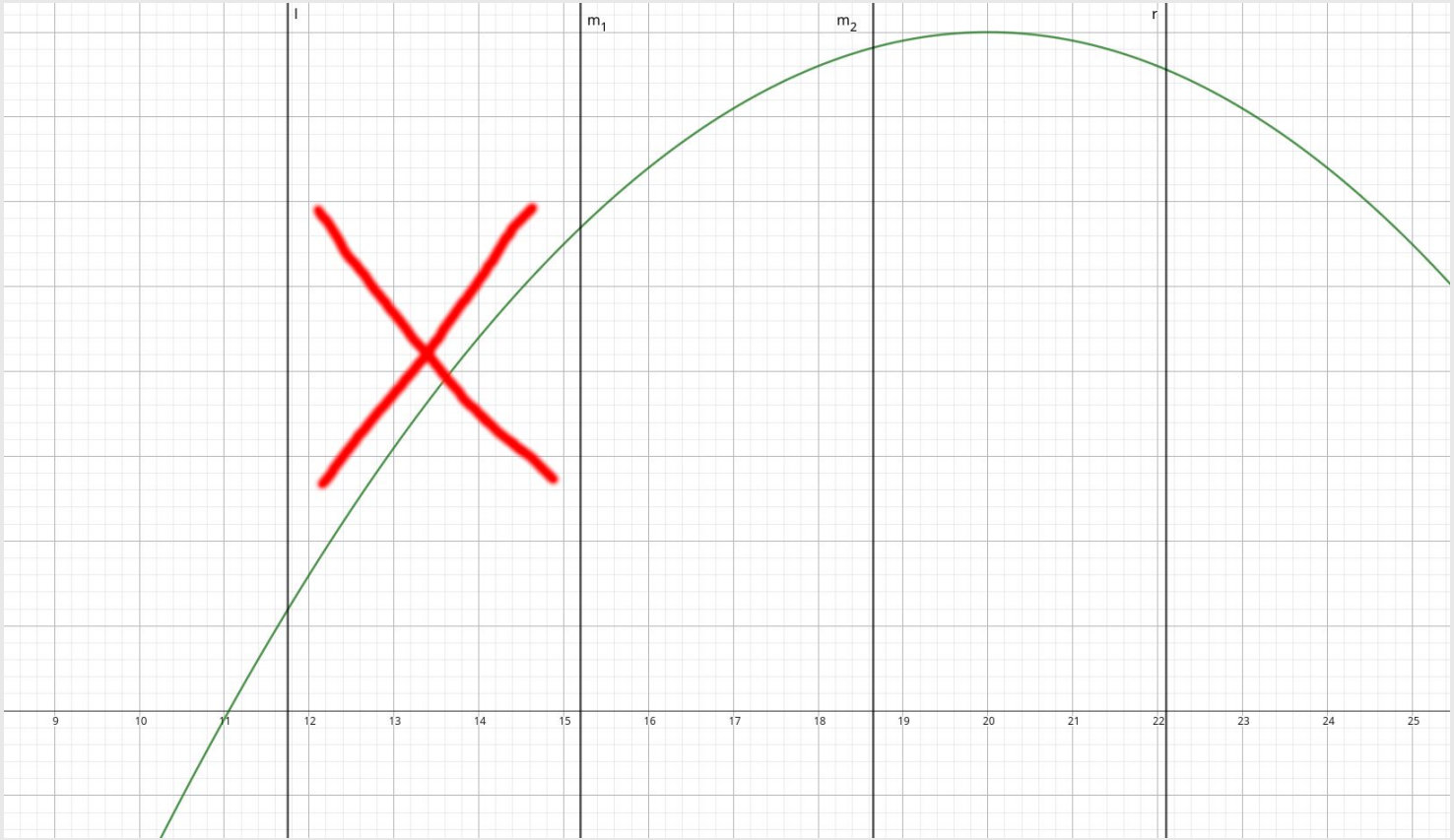
Busca Ternária



Busca Ternária



Busca Ternária



Busca Ternária

- A cada iteração:
  - Reduzimos o intervalo original para ⅔ do seu tamanho
- Quantas iterações até terminar a busca?
- Seja L o espaço (tamanho do intervalo [l, r] inicial) e k o número de iterações. A busca acaba quando:

$$L \cdot \left(\frac{2}{3}\right)^k \approx 1$$

- Logo:

$$k \approx log_{3/2}(L) = O(log(L))$$

- Problemas com double: iterar um número fixo de vezes ou até chegar na precisão desejada

Busca Ternária

Quando saber que podemos usar busca ternária?

- Intuição
- Convexidade
  - Várias funções na geometria são convexas (Ex.: $f(x) = x^2$)
  - Algumas operações preservam a convexidade
  - Sejam **f** e **g** funções convexas, as seguintes funções também são:
    - $h(x) = a*f(x)$, $a>=0$
    - $i(x) = f(x) + g(x)$
    - $j(x) = \max(f(x), g(x))$
  - Em alguns problemas, precisamos minimizar uma função convexa

Busca Ternária

Exemplo:
Encontre a menor distância de um ponto em uma reta (**a*x + b**) para um ponto fixo (**x, y**).

Busca Ternária

Exemplo:
Encontre a menor distância de um ponto em uma reta (**a*x + b**) para um ponto fixo (**x, y**).

Solução:
- Busca ternária na resposta.
- Um ponto na reta → (**x**, a*x + b)
  - A distância ao quadrado é uma função quadrática (convexa) em **x**

Complexidade:
- Avaliar a distância para um dado **x**: O(1)
- Complexidade final: O(log)

Busca Ternária

Exemplo:
Encontre a menor soma das distância de um ponto em uma reta (**a*x + b**) para **n** pontos (**xi, yi**), para algum ponto da reta.

Busca Ternária

Exemplo:
Encontre a menor soma das distância de um ponto em uma reta (**a*x + b**) para **n** pontos (**xi, yi**), para algum ponto da reta.

Solução:
- Busca ternária na resposta.
- Um ponto na reta → (**x**, a*x + b)
    - A soma das distâncias ao quadrado é uma função convexa em **x**

Complexidade:
- Avaliar a distância para um dado **x**: O(n)
- Complexidade final: O(n*log)

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Buscas Ternárias Aninhadas

Buscas Aninhadas

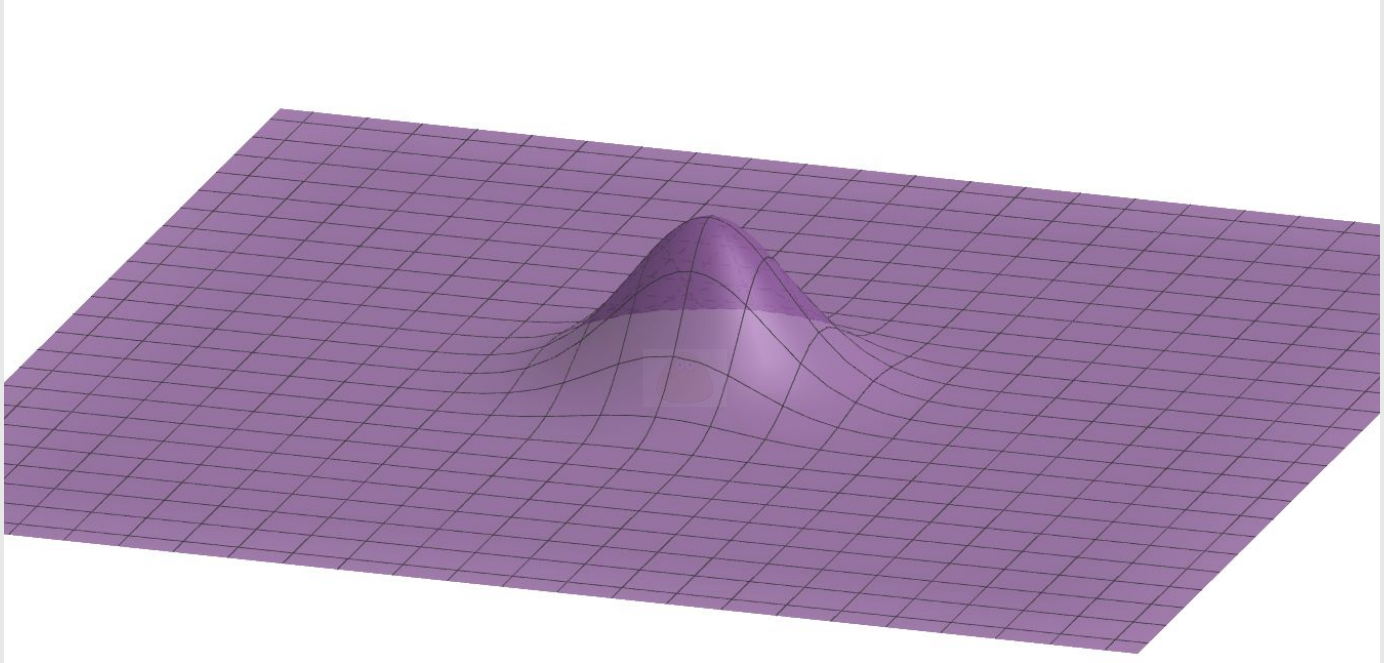
- É possível aninhar buscas ternárias
  - A função objetivo de uma busca ternária é o resultado de outra busca ternária
- Usada para resolver problemas de otimização multi-dimensionais
  - Otimiza uma função multi-dimensional unimodal

Buscas Aninhadas

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco



Busca Aninhadas

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

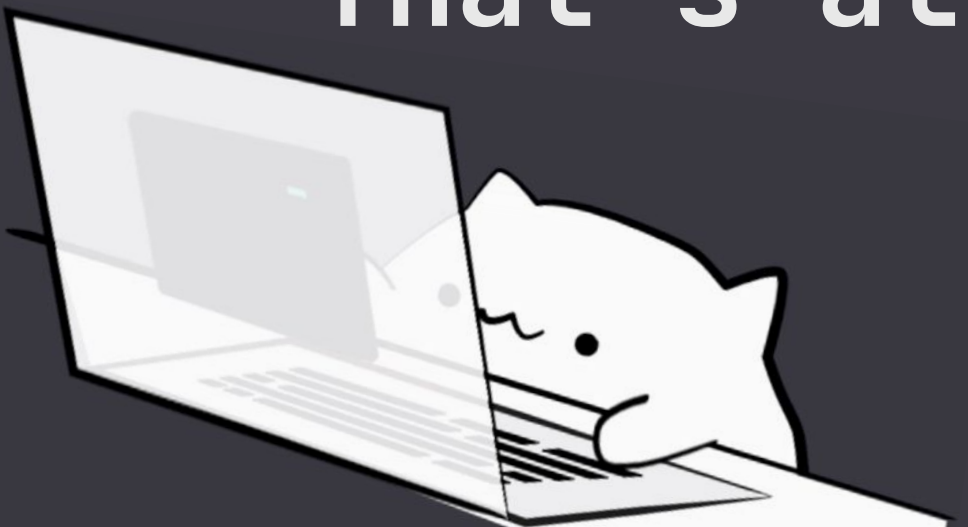
Exemplo:
Encontre a menor soma das distância de um ponto no plano (**x, y**) para **n** pontos (**xi, yi**), para qualquer ponto (**x, y**).

Solução:
- Busca ternária, de novo.
- Fazemos a busca no **x**, onde a função f, é uma busca ternária no **y**.
    - Cada busca ternária no **y** recebe um valor de **x** fixo e procura otimizar a soma das distâncias dentre os pontos na reta x=**x**.

Complexidade:
- Avaliar a distância para um dado x: O(n*log)
- Complexidade final: $O(n*\log^2)$

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

That’s all, Folks!